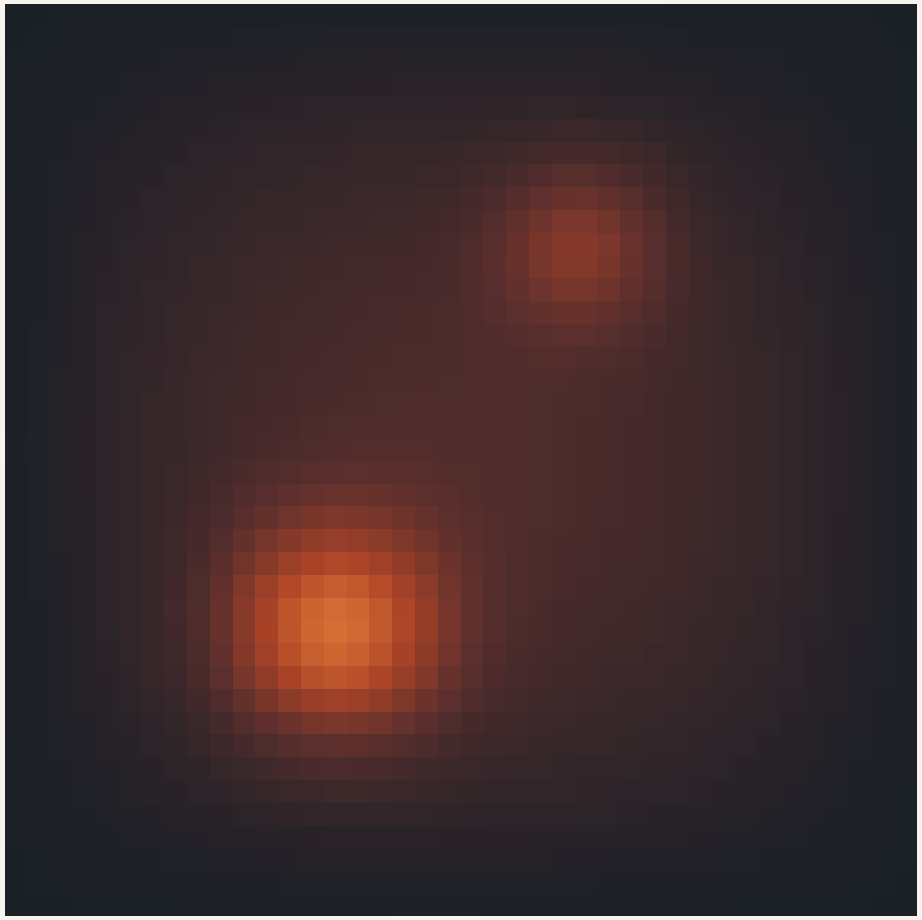
Heat diffusion, in parallel

Jacobi and conjugate gradient
for the same implicit heat equation

Research question

When does communication overtake computation?

2D backward Euler
1–64 MPI ranks · 280 measured runs



Measured temperature field

N = 128 · step 5

An implicit step is a linear solve

$$\partial T / \partial t = \alpha \nabla^2 T$$

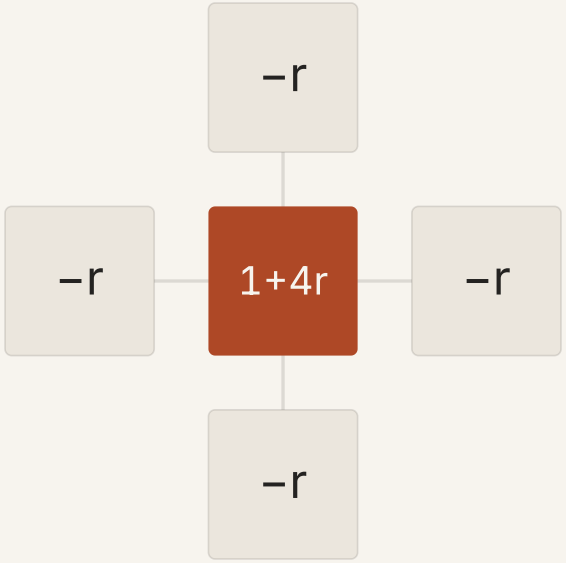
Unit square, zero-temperature edges
N × N interior points

$$(I - \alpha \Delta t \, L_h)\, x = b$$

b is the previous temperature field
x is the next temperature field

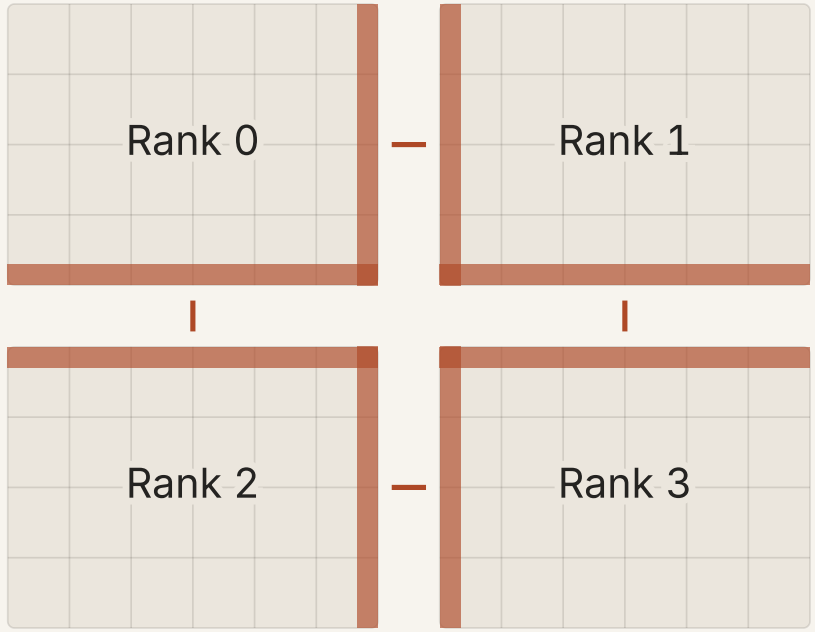
$$r = \alpha \Delta t / h^2$$

with h = 1 / (N + 1)



Matrix-free five-point stencil

Each rank owns a piece of the plate



Example: 2 × 2 Cartesian process grid

01 **Partition**

Rectangular blocks, including uneven sizes

02 **Exchange halos**

Neighbor values move with MPI_Sendrecv

03 **Combine scalars**

Dot products use MPI_Allreduce

Fewer iterations, more synchronization

Jacobi

$x_{new} = x + (b - Ax) / (1 + 4r)$

Local updates using the current residual

Halo exchange	Every update
Norm check	First, then every 10 updates
Typical tradeoff	More updates, fewer reductions

Conjugate gradient

Ap, $p^T Ap$, $r^T r$, then a new direction

Unpreconditioned CG on the same system

Halo exchange	Every update
Global dot products	Two per update
Typical tradeoff	Fewer updates, more synchronization

Both start from x = b and must pass $\|b - Ax\|_2 / \max(\|b\|_2, 10^{-30}) \le 10^{-8}$

280 runs on two exclusive AMD nodes

280

validated runs

56 configurations × 5 trials

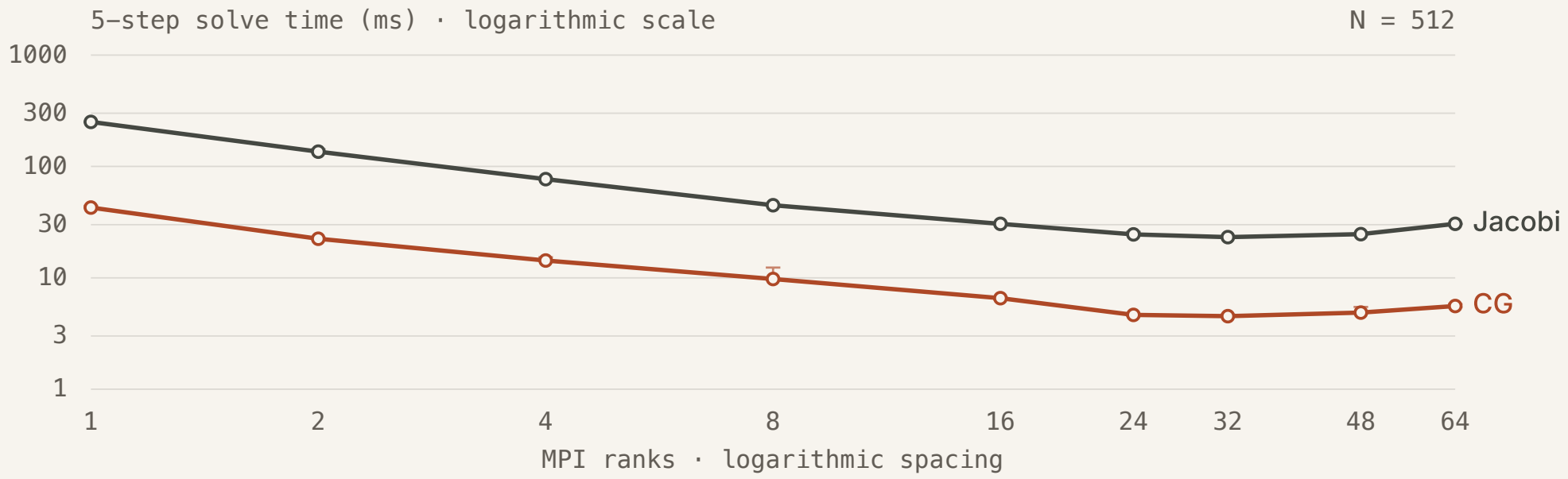
Five measured steps per run

Hardware	2 × AMD EPYC 7713 nodes
Strong scaling	N = 512, 1024 P = 1, 2, 4, 8, 16, 24, 32, 48, 64
Other studies	Weak scaling and r = 0.5, 2, 8
Measurement	Reset after warm-up Sum of each step’s slowest-rank time

P = 1 uses one node. P ≥ 2 uses both nodes with core binding. Fixed r = 2 for scaling.

CG is faster at every tested rank count

Strong scaling · fixed grid · five-step solve



5.13× faster

CG vs Jacobi at N = 512, P = 32

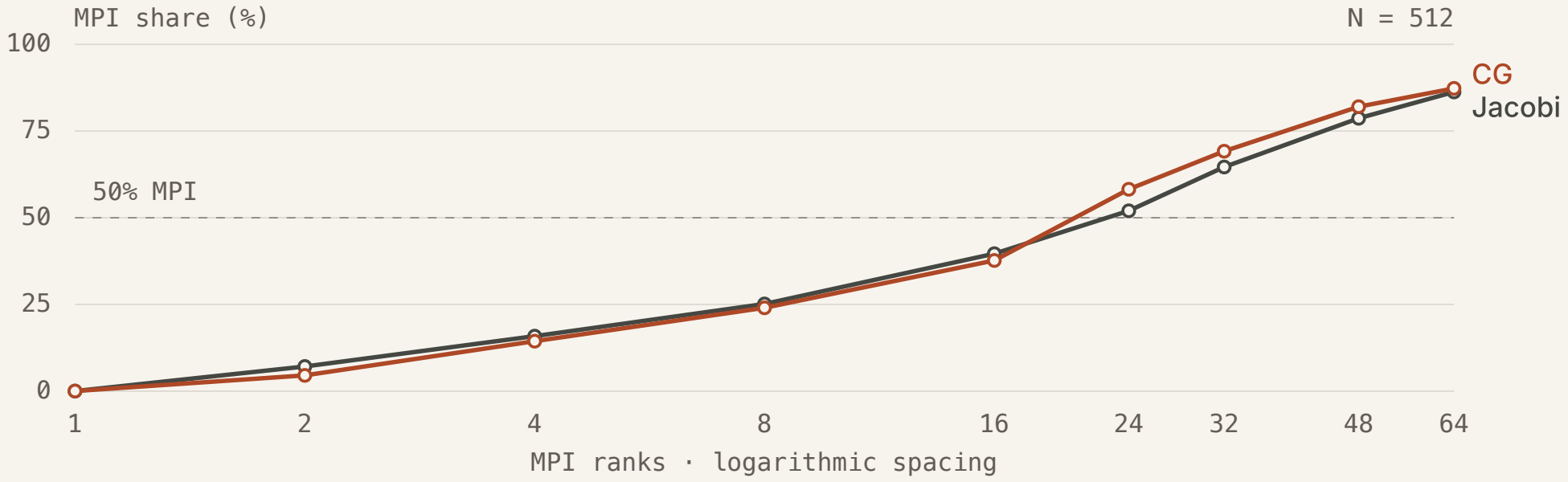
7.68× faster

CG vs Jacobi at N = 1024, P = 64

Medians of five trials
Whiskers show min–max

More ranks shift the balance toward MPI

Halo + reduction time, as a share of compute + MPI time



16–24 ranks

50% crossover bracket at N = 512

48–64 ranks

50% crossover bracket at N = 1024

Both solvers
MPI time includes waiting

CG wins by doing far fewer updates

N = 1024 · P = 64 · five steps

Jacobi	450
CG	24

18.75× fewer updates
7.68× lower median runtime

Each CG update costs more

More reductions and vector operations make each update more expensive

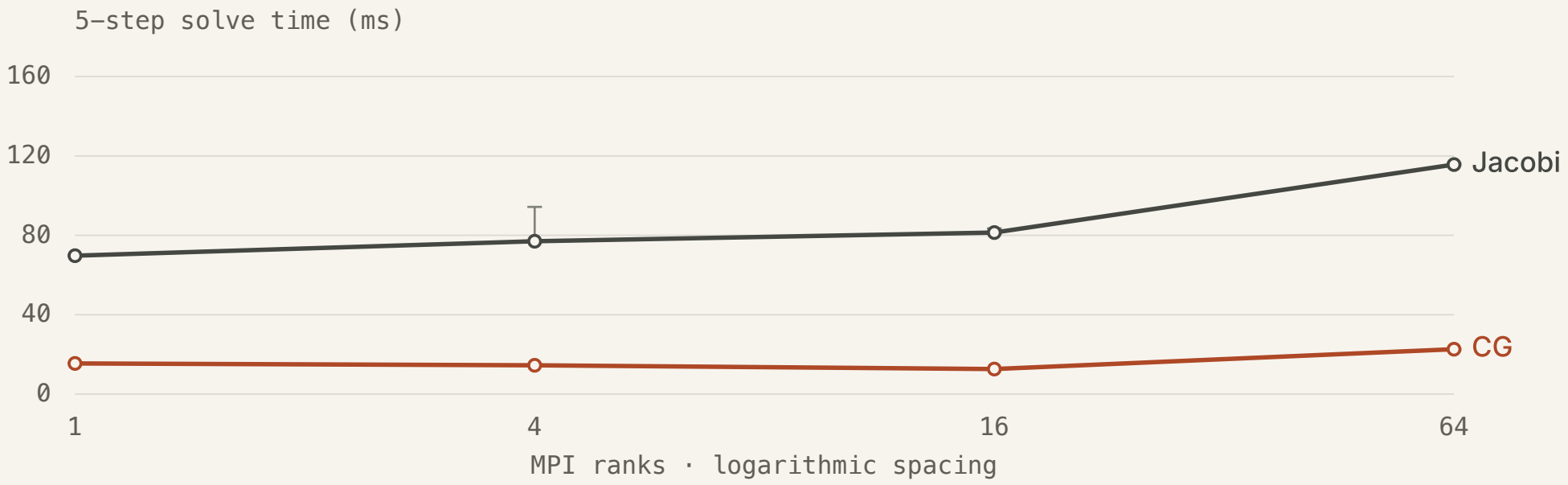
The update count still wins

CG reduces the total work enough to offset its higher cost per update

Observed for this initial field, tolerance, r, and implementation

Weak scaling reveals the cost per update

256 × 256 owned cells per rank · N grows from 256 to 2048



69.7 → 115.7 ms

Jacobi five-step runtime, P = 1 to 64

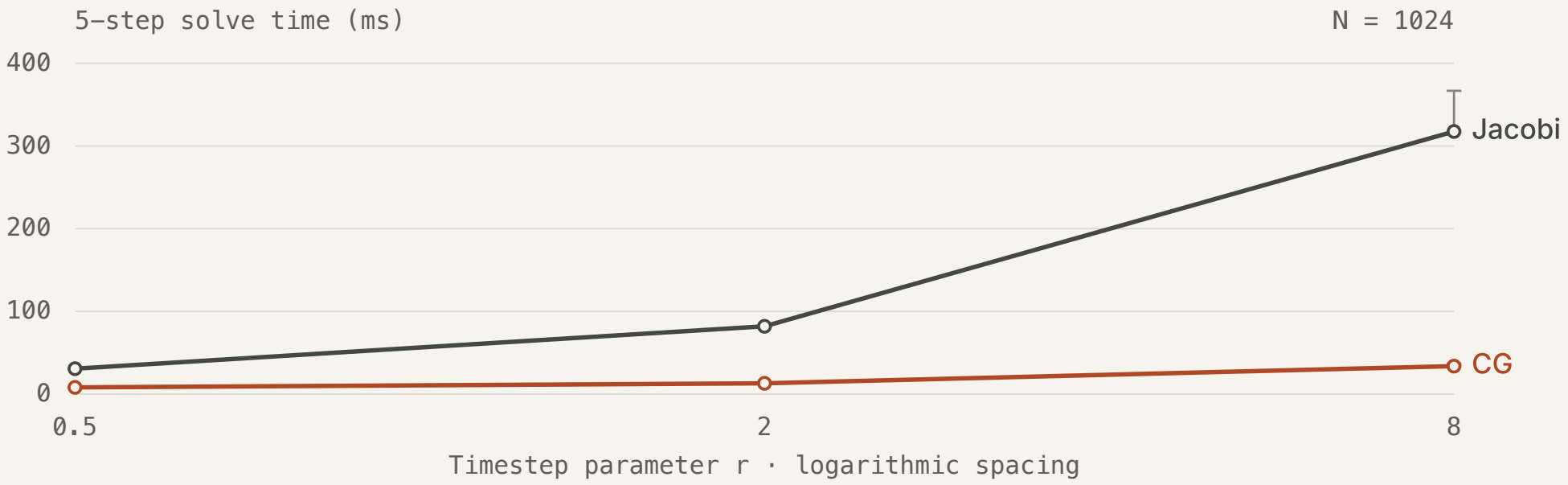
15.4 → 22.6 ms

CG five-step runtime, P = 1 to 64

Update counts fall with N
Total time conceals that change

Larger timesteps require more solver work

N = 1024 · P = 16 · r = αΔt / h²



150 → 1920

Jacobi updates as r increases from 0.5 to 8

11 → 85

CG updates over the same five steps

Physical Δt changes with r
Equal step counts, different durations

Validated solutions, bounded conclusions

Correctness checks

Independent dense serial reference

Analytic sine mode and refinement

Uneven blocks, halos, and boundaries

True-residual and timing diagnostics

Largest benchmark residual

9.985×10^{-9}

below the 10^{-8} tolerance

Interpretation limits

01 MPI time includes waiting and load imbalance

02 P = 1 uses one node
P ≥ 2 uses two

03 Five nearby trials share one allocation

04 Fixed r changes Δt with N
Physical durations differ

CG lowers runtime and MPI limits scaling

01 CG was faster at every tested strong-scaling configuration

02 Larger grids postpone communication dominance

03 Communication can dominate while runtime still improves

Reproduce the study

Build with CMake and OpenMPI
Run inside a Slurm allocation

```
mpirun -np 4 ./build/implicit_heat
--N 512 --steps 5 --solver cg
--r 2 --tol 1e-8 --output run.json
```

Build and run instructions

Full report

Measured summary CSV